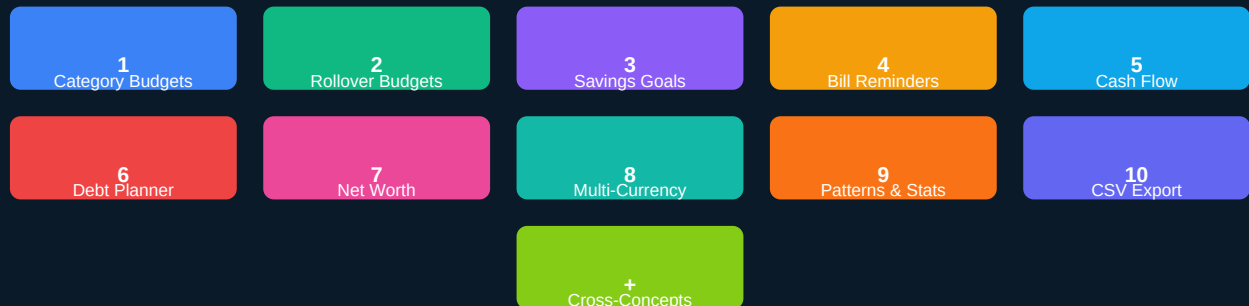


MONEYTRACKER · DEVELOPER REFERENCE

Budgeting Concepts

Everything you need to build the finance features

Formulas · Logic · Database design · Edge cases · Concepts



Contents

1	Category Budgets A spending limit per category, tracked against actual spending within a calendar month.	V2 Feature
2	Rollover Budgets Unspent budget from one month is added to the next month's allowance for that category.	V2 Feature
3	Savings Goals A named target amount with a running total of manual contributions and a projected completion date.	V2 Feature
4	Bill & Recurring Expense Reminders Fixed scheduled liabilities logged with due dates; local WorkManager notifications fire before each due date.	V2 Feature
5	Cash Flow Projections Anticipated income minus anticipated costs projected 1–3 months forward using historical averages and committed bills.	V3 Feature
6	Debt Payoff Planner Month-by-month amortisation simulation comparing Snowball and Avalanche payoff strategies.	V3 Feature
7	Net Worth Tracker Total assets minus total liabilities, tracked monthly as a historical snapshot series.	V3 Feature
8	Multi-Currency Support Transactions logged in any currency, converted to the user's base currency for all aggregates, with exchange rates set manually.	V3 Feature
9	Spending Pattern Analysis Deterministic statistical rules applied to transaction history to surface automatically generated observations.	V3 Feature
10	CSV Data Export Transaction history exported as a standard comma-separated values file to the user's chosen local folder.	V2 Feature
◆	Cross-Cutting Concepts Concepts that underpin multiple features — understand these once, apply them everywhere.	All Versions

1

Category Budgets

V2 Feature

Core concept: A spending limit per category, tracked against actual spending within a calendar month.

Remaining = Budget Limit – Amount Spent (in category, this month)

Progress % = (Spent ÷ Limit) × 100

Variance = Limit – Spent [positive = under budget, negative = over]

The Monthly Period

A budget is always scoped to a calendar month. Your Room Database queries must filter transactions by the current month AND year — not just month number. On the 1st of each month, the 'spent' total resets to zero automatically; the limit itself stays unchanged until the user edits it.

Three Visual States

The progress bar has three states based on the progress percentage:

- 0 – 79% → Green (healthy, on track)
- 80 – 99% → Amber (approaching limit — optional notification here)
- 100%+ → Red (limit exceeded — app does NOT block further spending)

Database Design

You need a separate table for budgets, not just transactions:

- budget_id, category_id, month, year, limit_amount
- Query transactions WHERE category_id = X AND month = M AND year = Y to get 'spent'
- Never hardcode the current month — derive it from a date parameter so historical views work

■ The app should never block a transaction because a budget is exceeded. Budgets are informational guardrails, not locks. Blocking spending creates user frustration and teaches distrust of the app.

Budget vs. Actual in Reports

For the reports screen, show both the budgeted amount and the actual spend side by side. A positive variance (under budget) is shown in green. A negative variance (over budget) in red. This is called a budget vs. actual comparison and is one of the most useful reports you can provide.

2

Rollover Budgets

V2 Feature

Core concept: Unspent budget from one month is added to the next month's allowance for that category.

Rollover Amount = $\max(0, \text{Last Month's Limit} - \text{Last Month's Spent})$

Next Month Effective Limit = $\text{Base Limit} + \text{Rollover Amount}$

Rollover Amount (capped) = $\min(\text{Rollover Amount}, \text{Rollover Cap})$

Why $\max(0, \dots)$?

You only roll over a surplus, not a deficit. If the user overspent last month ($\text{spent} > \text{limit}$), the rollover is zero — they don't carry a penalty forward. This keeps the system encouraging rather than punishing.

Cumulative vs. Single-Month Rollover

There are two ways to model this — choose one and be consistent:

- Single-month rollover: only the immediately prior month's surplus carries forward. Simple to implement.
- Running surplus balance: keep a cumulative surplus field per category that accumulates month over month. More powerful but requires careful migration if you change the model later.
- Recommended for V2: single-month rollover first. The cap prevents abuse.

The Rollover Cap

Without a cap, a category the user rarely uses (e.g. 'Medical') could accumulate an enormous rollover that makes the progress bar meaningless. A sensible default cap is $2\times$ the base limit. Store it as a user-configurable field per category: `rollover_cap` (nullable — null means no cap).

What to Store in the Database

You need a monthly snapshot table because you can't reconstruct rollover history from transactions alone:

- `budget_snapshot`: `category_id`, `month`, `year`, `base_limit`, `rollover_amount`, `effective_limit`, `actual_spent`
- Write this snapshot at the end of each month (or lazily when the next month is first opened)
- Reading history: query the snapshot table, not live transactions, for past months

■ Rollover is toggled per category. Do not apply it globally. Users want it for 'Entertainment' but probably not for 'Rent'. Store an `is_rollover_enabled` boolean per category budget row.

3

Savings Goals

V2 Feature

Core concept: A named target amount with a running total of manual contributions and a projected completion date.

Progress % = (Total Contributed ÷ Target Amount) × 100

Remaining = Target Amount – Total Contributed

Avg Monthly Contrib = Sum of contributions (last 3 months) ÷ 3

Months to Completion = Remaining ÷ Avg Monthly Contribution

Estimated Date = Today + Months to Completion

Contributions Are Not Transactions

A goal contribution is a separate concept from an expense or income transaction. It represents the user intentionally allocating money toward a goal — it does not map to a spending category. Store contributions in their own table: goal_id, date, amount, note.

Handling the Completion Estimate Edge Cases

The formula breaks in several scenarios — handle each:

- Average = 0 (no contributions in 3 months): show 'No estimate — start contributing' instead of a date
- Less than 3 months of history: use however many months are available (1 or 2)
- Target already reached: show 'Goal achieved!' and move to Completed state
- Estimated date is in the past (due to contributions slowing down): just show the honest projected date

Goal States

A goal moves through three states — store this as a status enum:

- ACTIVE: in progress, contributions ongoing
- COMPLETED: Total Contributed >= Target Amount — archive, do not delete
- ARCHIVED: user manually dismissed without completing — hide from main list but keep in history

Database Design

- goals table: goal_id, name, target_amount, created_date, target_date (nullable), status
- goal_contributions table: contribution_id, goal_id, date, amount, note
- Total contributed = SUM(amount) FROM goal_contributions WHERE goal_id = X

■ Completed goals should feel like achievements. When a goal is completed, show a celebration animation and archive it to a 'Completed Goals' section the user can revisit. This positive reinforcement is a key part of the motivation loop.

4

Bill & Recurring Expense Reminders

V2 Feature

Core concept: Fixed scheduled liabilities logged with due dates; local WorkManager notifications fire before each due date.

Fixed vs. Variable Expenses

In budgeting, all expenses split into two types. Fixed expenses recur predictably on a known date: rent, EMIs, subscriptions, insurance premiums. Variable expenses are discretionary and irregular: food, entertainment, shopping. This feature handles fixed expenses exclusively. Understanding this distinction matters because fixed expenses are also the foundation of cash flow projections in V3.

Recurrence Patterns

The most common patterns you need to support:

- Monthly (same date each month) — most common: rent, EMIs, subscriptions
- Weekly — gym, weekly transfer, etc.
- Bi-weekly — some loan repayments
- Annual — insurance premiums, domain renewals
- Custom N days — for anything else

Notification fires at: $\text{due_date} - \text{lead_days}$

Next due date (monthly): same day number, next month

Monthly cost (weekly bill): $\text{weekly_amount} \times 4.33$

Monthly cost (annual bill): $\text{annual_amount} \div 12$

Total committed monthly: SUM of all active bills' monthly equivalents

Edge Case: End-of-Month Due Dates

A bill due on the 31st has no valid date in February, April, June, etc. Standard approach: snap to the last valid day of the month. So a 31st-due bill in February 2027 fires on the 28th. Store the original due day (31) and compute the actual date at scheduling time.

WorkManager Implementation Notes

Android WorkManager is the correct tool — not AlarmManager or Handler:

- Use OneTimeWorkRequest per upcoming reminder, not PeriodicWorkRequest
- After each reminder fires (or bill is marked paid), schedule the next OneTimeWorkRequest for the next recurrence
- WorkManager survives device restarts automatically — this is its key advantage
- Reschedule all pending reminders on BOOT_COMPLETED as a safety net

The Committed Cost Concept

Sum the monthly-equivalent of every active recurring bill to produce a single 'Committed Monthly Expenses' figure. This is distinct from discretionary spending. Showing the user this number — e.g. 'Your committed monthly expenses total ■18,400' — immediately clarifies how much of their income is already spoken for.

- Offer an option to auto-log a bill as a transaction when the user marks it 'Paid'. This connects the reminder system to the transaction history seamlessly and reduces double-entry for users who track recurring expenses manually.
-

5

Cash Flow Projections

V3 Feature

Core concept: Anticipated income minus anticipated costs projected 1–3 months forward using historical averages and committed bills.

Projected Net (month N) = Projected Income – Committed Costs – Discretionary Estimate

Discretionary Estimate = (Month-1 spend + Month-2 spend + Month-3 spend) ÷ 3

Projected Closing Bal. = Current Balance + Projected Net

Three Layers of a Projection

Each month's projection is built from three layers with different certainty levels:

- Layer 1 — Committed Income: salary or recurring income the user has logged. HIGH certainty.
- Layer 2 — Committed Expenses: all active recurring bills from the Reminders feature. HIGH certainty.
- Layer 3 — Discretionary Spending: trailing 3-month category averages. LOWER certainty.

The Trailing Average in Detail

For each discretionary category, sum the total spent in each of the last 3 complete calendar months, then divide by 3. Use only complete months — the current partial month is excluded. If a category has fewer than 3 months of history, use however many complete months are available.

The Waterfall Chart Concept

A waterfall chart is the clearest way to visualise a cash flow projection. It starts with an opening balance bar, adds income as a rising segment, then subtracts each cost as a falling segment, arriving at a projected closing balance. Each step is labelled with the category and amount.

- Opening balance (current actual)
- + Projected income (green, rising)
- – Committed bills (amber, falling)
- – Discretionary categories (red/grey, falling)
- = Projected closing balance (teal if positive, red if negative)

Representing Decreasing Confidence

Month 1 is more reliable than month 3. Do not show all months with equal visual weight. Options: fade opacity for months further out; show a range band instead of a single bar (e.g. ±15% around the estimate); add a note like 'estimate becomes less reliable beyond 1 month'.

■ Never label projections as predictions or guarantees. Use language like 'estimated', 'based on recent patterns', or 'projected'. Users make real financial decisions based on this — be clear about uncertainty.

6

Debt Payoff Planner

V3 Feature

Core concept: Month-by-month amortisation simulation comparing Snowball and Avalanche payoff strategies.

How Debt Amortisation Works

Each month, a portion of the payment goes to interest and the rest reduces the principal. Early in the loan, most goes to interest. As the balance shrinks, less goes to interest and more to principal — this is why payoff accelerates toward the end.

Monthly Interest Charge = Balance × (Annual Rate ÷ 12)

Principal Reduction = Monthly Payment – Monthly Interest Charge

New Balance = Balance – Principal Reduction

(repeat each month until Balance ≤ 0)

Snowball Method

Pay the minimum on every debt. Direct all extra payment capacity toward the debt with the smallest outstanding balance. When that debt is cleared, redirect its minimum payment + the extra toward the next smallest balance.

- Psychological advantage: debts disappear quickly, creating motivation
- Financial disadvantage: may pay more total interest than Avalanche
- Best for users who need wins to stay motivated

Avalanche Method

Pay the minimum on every debt. Direct all extra toward the debt with the highest annual interest rate. When that's cleared, redirect to the next highest rate.

- Financial advantage: minimises total interest paid over the entire payoff
- Psychological disadvantage: if the highest-rate debt is also large, progress feels slow
- Best for users who are analytically motivated and can stay the course

The Simulation Algorithm

Run this loop for each strategy until all balances reach zero:

- Step 1: For each debt, compute monthly interest and subtract minimum payment
- Step 2: Apply the extra payment to the target debt (smallest or highest-rate depending on strategy)
- Step 3: If a debt reaches zero, mark it cleared — redirect its minimum to the next target
- Step 4: Record (month_number, debt_name, remaining_balance) for each debt
- Step 5: Repeat until all debts are zero

Total Interest (Snowball) = SUM of all interest charges across simulation

Total Interest (Avalanche) = SUM of all interest charges across simulation

Interest Saved = Snowball Total Interest – Avalanche Total Interest

Debt-Free Date = Today + Total Months in Simulation

Key Outputs to Show the User

Side by side for both strategies:

- Total months to debt-free
- Total interest paid
- Projected debt-free date
- The delta between strategies: 'Avalanche saves ■X in interest, takes Y months longer/shorter'

■ The delta between strategies is the single most valuable piece of information you can give the user. It makes the abstract concept of 'interest' concrete: 'Following Avalanche saves you ■8,400 and clears debt 3 months faster.'

7

Net Worth Tracker

V3 Feature

Core concept: Total assets minus total liabilities, tracked monthly as a historical snapshot series.

Net Worth = Total Assets – Total Liabilities

Net Worth Change = This Month's Net Worth – Last Month's Net Worth

What Counts as an Asset

Assets are things you own that have monetary value:

- Savings accounts (current balance — user enters manually)
- Investments (current market value — user estimates)
- Physical assets: phone, laptop, vehicle (user-estimated resale value)
- Property (current estimated value)
- Other (anything else with value)

What Counts as a Liability

Liabilities are amounts owed to others — always use the outstanding balance, not the original loan amount:

- Outstanding personal loan balance
- Credit card dues currently unpaid
- EMI remainder (how much of the loan is still unpaid, not the monthly EMI amount)
- Education loans, home loans, other debts

The Snapshot Model

Unlike transactions logged in real time, net worth is updated periodically. Each update creates a timestamped snapshot. The historical graph is built from this snapshot series — you are not deriving net worth from transactions, you are recording it directly as a point-in-time observation.

- net_worth_snapshots table: snapshot_id, date, total_assets, total_liabilities, net_worth
- asset_entries table: entry_id, snapshot_id, name, category, value
- liability_entries table: entry_id, snapshot_id, name, category, balance
- Typical cadence: user updates once per month at month-end

Net Worth Can Be Negative — That Is Fine

A student with education loans and few assets will have a negative net worth. Display this honestly — negative in red — and emphasise the trend over absolute value. A net worth moving from –■200,000 to –■180,000 is real progress.

Why This Complements Expense Tracking

Expense tracking = cash flow (monthly money in vs. money out). Net worth = wealth accumulation (the cumulative result of all financial behaviour). You can have great cash flow but stagnant net worth (if you're spending all savings). You can have poor cash flow one month but rising net worth (if an investment grew). Together they give the complete picture.

■ Asset values depreciate. A phone worth ■50,000 today is worth less next month. Do not try to calculate depreciation automatically — this adds complexity and is often inaccurate. Simply let the user update asset values when they review their net worth. Manual accuracy is better than automated approximation here.

8 Multi-Currency Support

V3 Feature

Core concept: Transactions logged in any currency, converted to the user's base currency for all aggregates, with exchange rates set manually.

Base Amount = Foreign Amount × Exchange Rate
 (where Exchange Rate = 'how many base units equal one foreign unit')
 e.g. 1 USD = 83.50 INR → \$45 × 83.50 = ■3,757.50

What to Store Per Transaction

Store both the original currency and the base equivalent on every transaction:

- original_amount: the amount as the user typed it
- original_currency: the 3-letter ISO currency code (USD, EUR, GBP, etc.)
- base_amount: original_amount × exchange_rate_used
- exchange_rate_used: the rate at the time of logging (frozen for this transaction)
- Use base_amount in all aggregates, reports, and totals

Why You Freeze the Rate at Transaction Time

Exchange rates fluctuate. If you always converted using the current rate, your historical reports would silently change every time the user updates their rates. A transaction logged at 1 USD = 83.50 INR should always show ■3,757.50 — not a different number next month when the rate is 84.20. Freezing the rate preserves the integrity of historical data.

Rate Updates Are Prospective Only

When the user updates an exchange rate, it only applies to new transactions logged after that update. Old transactions keep the rate they were logged with. Never retroactively recalculate base_amount for existing transactions.

Database Design

- currencies table: currency_code (PK), currency_name, exchange_rate, is_base (boolean)
- Only one currency has is_base = true
- transactions table gains: original_amount, original_currency_code, exchange_rate_used
- base_amount is either stored (faster queries) or computed (less storage)

UI Considerations

In the transaction list, show both values for foreign transactions:

- Primary display: ■3,757.50 (base amount — used in all totals)
- Secondary display: \$45.00 @ 83.50 (original amount + rate)
- In reports and totals: always use base_amount only

■ Do not offer to fetch live exchange rates. This would require INTERNET permission. Make the manual rate entry as frictionless as possible instead — pre-populate the field with the last used rate for that currency.

9

Spending Pattern Analysis

V3 Feature

Core concept: Deterministic statistical rules applied to transaction history to surface automatically generated observations.

The Three Core Statistical Concepts

This feature is built on just three ideas: the average, variance, and trend.

Trailing Average = $\text{SUM}(\text{monthly spend in category, last } N \text{ months}) \div N$

Variance % = $((\text{This Month} - \text{Average}) \div \text{Average}) \times 100$

Trend = $\text{Recent 3-month avg} - \text{Prior 3-month avg}$

[positive = spending increasing, negative = decreasing]

Weekend vs. Weekday Spending Split

Filter transactions by day of week, then compare daily averages:

- Weekend total = SUM of transactions on Saturday + Sunday
- Weekday total = SUM of transactions Monday through Friday
- Weekend daily avg = Weekend total ÷ Number of weekend days in period
- Weekday daily avg = Weekday total ÷ Number of weekday days in period
- Ratio = Weekend daily avg ÷ Weekday daily avg

Ratio = $\text{Weekend Daily Average} \div \text{Weekday Daily Average}$

Ratio > 1.0 → spend more on weekends (common)

Ratio of 2.1 means: 'You spend 2.1× more on weekend days than weekdays'

Insight Thresholds — Filtering Noise

Not every variance is worth surfacing. A $\pm 5\%$ change is normal fluctuation. Only generate an insight if the observation crosses a meaningful threshold:

- Variance insight: only if $|\text{variance \%}| > 15\%$ (suggest $\pm 20\%$ as default)
- Trend insight: only if trend $> \pm \$500$ or $> 10\%$ of the base average
- Weekend/weekday ratio: only if ratio > 1.5 or < 0.67 (i.e. 50% more or less)
- Minimum data: skip a category if it has fewer than 2 months of history

Example Insight Strings

Template your insight messages — don't hardcode them:

- 'Your {category} spending is up {variance}% vs your average'
- 'You spent {amount} on {category} in {month} — your highest in {N} months'
- 'Your weekend spending is {ratio}x your weekday average'
- '{category} spending has increased for {N} months in a row'

■ Limit insights to 3–5 at a time. More than that feels overwhelming. Rank them by magnitude (largest variance first) and show only the top few. Let the user dismiss insights they don't find useful.



10 CSV Data Export

V2 Feature

Core concept: Transaction history exported as a standard comma-separated values file to the user's chosen local folder.

Standard Ledger Format

A CSV export of transactions should follow a clear, universally readable format:

Date	Amount	Currency	Type	Category	Note
2026-04-13	450.00	INR	Expense	Food	Lunch
2026-04-13	15000.00	INR	Income	Salary	April salary
2026-04-12	45.00	USD	Expense	Shopping	Amazon

Format Conventions

- Date: ISO 8601 format (YYYY-MM-DD) — sorts correctly in any spreadsheet, no ambiguity
- Amount: always positive — use the Type column (Expense/Income) to distinguish direction
- Currency: 3-letter ISO code (INR, USD, EUR). If multi-currency is off, all rows are base currency
- Type: 'Expense' or 'Income' — consistent casing
- Category: the user's category name exactly as they named it
- Note: the user's free-text note, or empty string if none

Android Storage Access Framework (SAF)

On Android, writing a file to user-chosen storage requires SAF — not a direct file path. Use `Intent(Intent.ACTION_CREATE_DOCUMENT)` to let the user pick a save location. The returned URI is then opened with `contentResolver.openOutputStream` to write the CSV bytes.

Generating CSV in Kotlin

Build the CSV as a string, then write to the URI:

- Header row: Date,Amount,Currency,Type,Category,Note
- For each transaction: join fields with commas, wrap notes containing commas in double-quotes
- Escape any double-quotes inside note text by doubling them: " → ""
- Write with UTF-8 encoding (important for ■ and other currency symbols)

Filter Options to Offer

- All time (full history)
- This month
- Last 3 months
- Custom date range (start date + end date picker)
- Optional: single-category export

■ Add a `WRITE_EXTERNAL_STORAGE` permission note in onboarding. On Android 10+ (API 29+), SAF does not require this permission for user-selected locations — but be aware of the API level split when testing.



Cross-Cutting Concepts

[All Versions](#)

Core concept: Concepts that underpin multiple features — understand these once, apply them everywhere.

Zero-Based Budgeting

In zero-based budgeting, every unit of income is assigned a purpose at the start of the month — a spending category, a savings goal, or a debt payment — so that income minus all allocations equals zero. MoneyTracker does not enforce ZBB, but category budgets + savings goals + bill reminders together support it for users who want to use it.

Fixed vs. Variable Expenses

This distinction matters across four features:

- Bill Reminders: tracks fixed expenses by definition
- Cash Flow Projections: treats fixed = committed, variable = estimated from history
- AI Savings Recommendations (V4): suggests cuts to variable, never to fixed
- Spending Pattern Analysis: variance analysis is more meaningful on variable categories

The 50 / 30 / 20 Guideline

A popular personal finance rule of thumb: allocate 50% of take-home income to needs (rent, food, transport, bills), 30% to wants (entertainment, dining out, shopping), and 20% to savings and debt repayment. MoneyTracker does not enforce this, but it is useful context when designing category defaults, budget suggestion prompts, and the AI savings recommendation logic in V4.

Discretionary vs. Non-Discretionary

Non-discretionary: necessary expenses with little control — rent, utility bills, loan EMIs, food staples, transport to work. Discretionary: optional choices — dining out, entertainment, shopping, subscriptions. The AI in V4 should only suggest reducing discretionary categories. Recommending a cut to rent is unhelpful and will damage trust in the feature.

Cash Flow vs. Net Worth — Two Views of Financial Health

Understand the distinction — both are important but measure different things:

- Cash Flow = money in minus money out for a period. Monthly. Can be positive even when you have high debt.
- Net Worth = total assets minus total liabilities. Cumulative. Reflects the result of years of behaviour.
- Good cash flow + rising net worth = genuinely improving financial position
- Good cash flow + flat net worth = earning well but not building wealth
- Poor cash flow + rising net worth = short-term squeeze but long-term progress (e.g. during loan repayment)

The Concept of Runway

Financial runway is how many months you could survive on current savings if income stopped. $\text{Runway} = \text{Liquid Assets} \div \text{Monthly Expenses}$. This is closely related to the emergency fund concept (3–6 months of runway is the common guideline). While MoneyTracker doesn't surface this explicitly, it is the conceptual foundation behind savings goals and net worth tracking.

■ When implementing the AI savings recommendations in V4, the logic should follow this priority order: (1) cover committed expenses first, (2) meet minimum debt payments, (3) contribute to emergency fund, (4) then allocate discretionary. Any recommendation that violates this order will feel wrong to users who know their finances well.

Formula Quick Reference

All key formulas from this guide in one place — keep this page open while building.

Feature	Formula / Rule
Category Budgets	$\text{Remaining} = \text{Limit} - \text{Spent}$ $\text{Progress \%} = (\text{Spent} \div \text{Limit}) \times 100$
Rollover Budgets	$\text{Rollover} = \max(0, \text{Last Limit} - \text{Last Spent})$ $\text{Next Limit} = \text{Base Limit} + \text{Rollover}$
Savings Goals	$\text{Progress \%} = (\text{Contributed} \div \text{Target}) \times 100$ $\text{Months left} = \text{Remaining} \div \text{Avg Monthly Contrib}$
Bill Reminders	Notification at: $\text{due_date} - \text{lead_days}$ $\text{Committed monthly} = \text{SUM of all bills' monthly equivalents}$
Cash Flow	$\text{Projected Net} = \text{Income} - \text{Committed} - \text{Discretionary}$ $\text{Discretionary} = \text{3-month trailing average per category}$
Debt — Amortisation	$\text{Monthly Interest} = \text{Balance} \times (\text{Annual Rate} \div 12)$ $\text{Principal Reduction} = \text{Payment} - \text{Monthly Interest}$
Debt — Strategies	Snowball: target smallest balance first Avalanche: target highest interest rate first
Net Worth	$\text{Net Worth} = \text{Total Assets} - \text{Total Liabilities}$ $\text{Change} = \text{This Month NW} - \text{Last Month NW}$
Multi-Currency	$\text{Base Amount} = \text{Foreign Amount} \times \text{Exchange Rate}$ Freeze rate at transaction time — never retroactively update
Patterns	$\text{Variance \%} = ((\text{This Month} - \text{Avg}) \div \text{Avg}) \times 100$ $\text{Trend} = \text{Recent 3-month avg} - \text{Prior 3-month avg}$
Weekend Ratio	$\text{Weekend daily avg} = \text{Weekend total} \div \text{Weekend days}$ $\text{Ratio} = \text{Weekend daily avg} \div \text{Weekday daily avg}$
CSV Export	Format: Date(ISO),Amount,Currency,Type,Category,Note Amounts always positive — Type column distinguishes direction